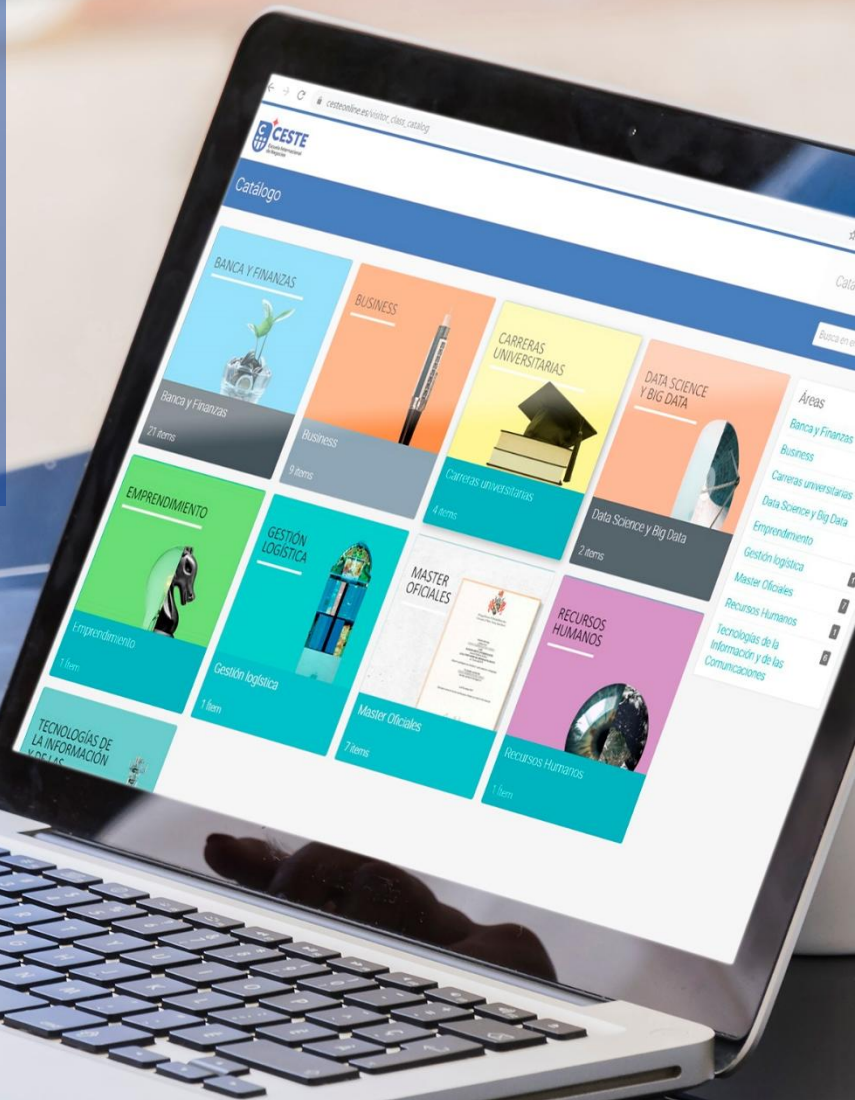


CESTE

Escuela Internacional de Negocios

Zaragoza (España)



Programa de hoy - Sesión 2

1. Delta Lake – Advance Features II

1. Partitioning
2. Z-order (limitaciones)
3. Liquid Clustering
4. DEMO I

2. Kahoot

3. Metastore and Tables

1. Data Plane vs Control Plane
2. Data & Metadata
3. External Tables vs Managed Tables
4. DEMO II

4. Set up Delta Tables

1. CTAs statements
2. Table Constraints
3. Cloning Delta Lake Tables
4. Views in Databricks
5. DEMO III

5. Data Processing and Transformation

1. Querying Files Directly
2. Extract Files as Raw Content
3. Configure Options for External Sources
4. CTAS from Files
5. DEMO IV
6. DEMO V

Objetivos de la Sesión



1. Avanzar en el entendimiento del concepto de layout de archivos.
2. Entender las técnicas que tenemos para interactuar con el Layout y sus limitaciones
3. Conocer el conjunto de queries y creación de tablas

Delta Lake – Advance Features II

1. Partitioning
2. z-order
3. Liquid Clustering



Data File Layout

Data File Layout

Define cómo se organizan y almacenan los ficheros que componen una tabla Delta. Una buena organización permite a Delta Lake **leer solo los ficheros relevantes**, mejorando notablemente el rendimiento de las consultas.

¿Por qué es importante?

- Permite aplicar data skipping
- Evita leer ficheros innecesarios
- Reduce el tiempo de ejecución
- Reduce el consumo de CPU
- Reduce los costes computacionales

Resultado

- Menos I/O
- Consultas más rápidas
- Mejor escalabilidad en tablas grandes

Estrategias:

- Partitioning
- Z-Order Indexing
- Liquid Clustering

Partitioning

Partitioning (Hive-style) organiza una tabla agrupando filas que comparten los mismos valores en columnas de partición.

Partitioning Column
↓

id	name	month	year	
1	Adam	9	2015	Partition 1 year=2015
2	Sarah	11	2015	
3	John	5	2000	Partition 2 year=2000
4	Ali	9	2000	
5	Emma	10	2000	
6	Eric	1	2023	Partition 3 year=2023
7	Sophia	3	2023	

Cada partición se almacena en un directorio físico independiente.

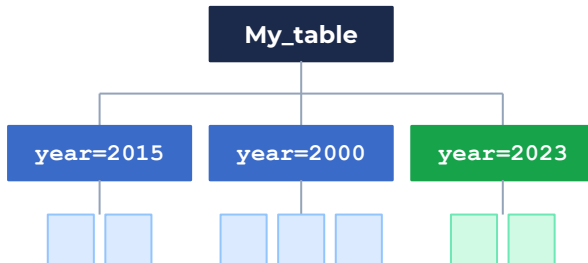
Al insertar datos, Delta Lake:

- Crea automáticamente los directorios
- Envía cada fila a su partición correspondiente

Partitioning

SQL

```
CREATE TABLE my_table (id INT, name STRING,  
year INT)  
PARTITIONED BY (year)
```



Operativa:

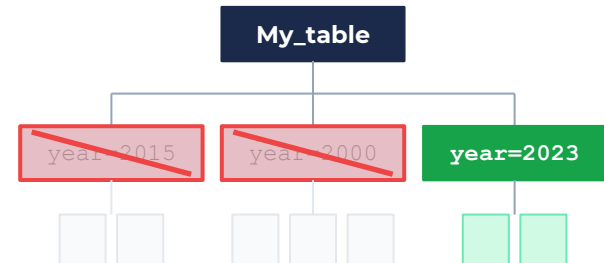
Cada partición se almacena en un directorio físico independiente.

Al insertar datos, Delta Lake:

crea los directorios automáticamente y envía cada fila a su partición correspondiente.

SQL

```
SELECT * FROM my_table  
  
WHERE year = 2023
```



Beneficio principal

Si una consulta filtra por una columna de partición, las particiones que no cumplen la condición se omiten completamente.

Resultado: menos datos leídos · consultas y operaciones más rápidas.

Delta Lake — Partitioning

⚠ Limitaciones importantes

Solo aporta beneficios en **tablas Delta muy grandes**. En tablas pequeñas o medianas:

- Se generan **muchos ficheros pequeños**
- OPTIMIZE actúa solo a **nivel de partición**
- Puede empeorar el problema de *small files*

✗ Malas prácticas

Particionar por columnas de **alta cardinalidad**:

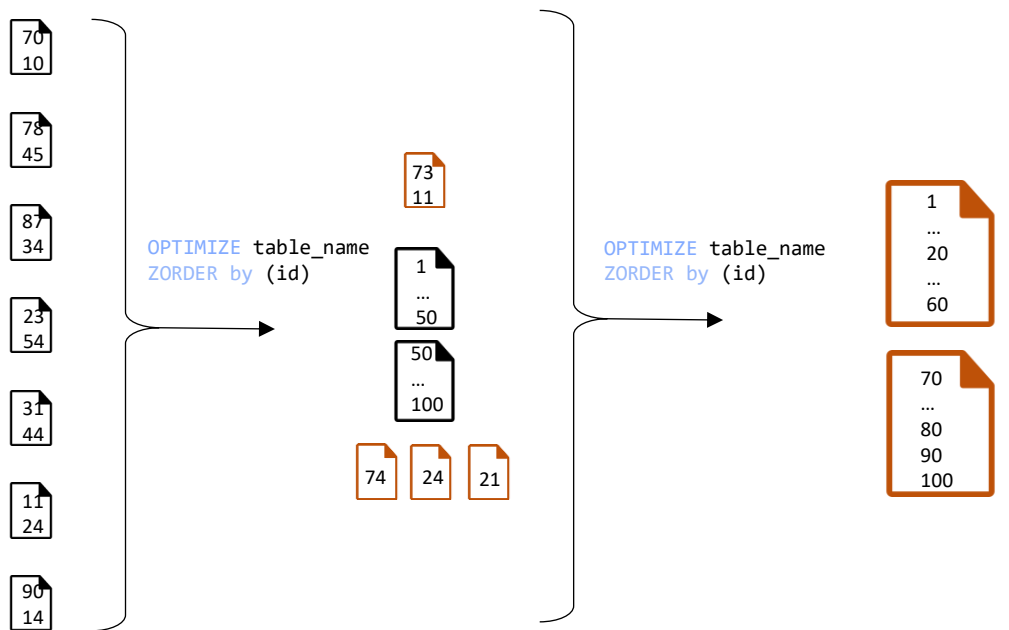
`user_id`, `transaction_id`, etc. Provoca:

- Demasiadas particiones
- Rendimiento ineficiente

Consideración clave Si la necesidad de la empresa cambia y requiere modificar la clave de partición: cambiar columnas de partición **requiere reescribir la tabla completa** — operación costosa en tiempo y recursos.

Z-Order (limitación)

Z-Order: Podemos usar Z-Order para tablas pequeñas pero tiene el problema que no es incremental al agregar nuevos datos.



⚠ Limitación principal

No es incremental

Cada nueva ingesta de datos:

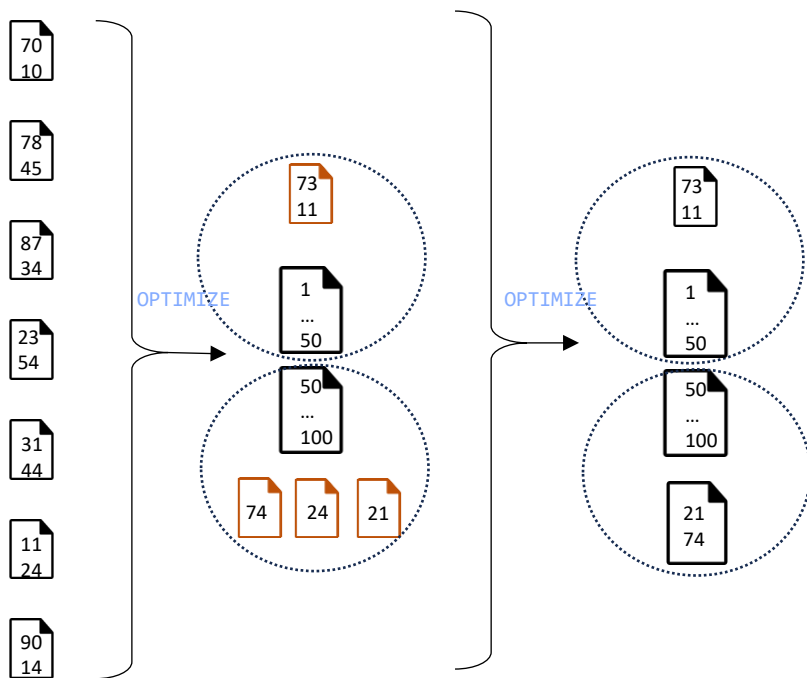
1. Requiere Volver a ejecutar Optimize
2. Reorganiza potencialmente toda la tabla

Puede ser:

1. Costoso computacionalmente
2. Lento en Tablas grandes

Liquid Clustering

Liquid Clustering: Es una evolución de Z-Order Indexing que ofrece más flexibilidad y mejor rendimiento, además de un comportamiento incremental como partitioning pero sin sus problemas.



SQL

```
ALTER TABLE my_table  
CLUSTER BY id
```

Características Clave:

1. Se define a nivel de Tabla
2. Usa Clustering Keys
3. No require especificarlas en cada OPTIMIZE
4. No crea subdirectorios específicos

Como resultado, las operaciones de optimización en Tablas Clusterizadas son rápidas y eficientes

Z-Order vs Liquid Clustering

Physical file organization strategies in Delta Lake

STEP 1 Raw Delta Table

Files written in Spark task order

id: 820	id: 12
id: 445	id: 91
id: 670	id: 303
id: 558	id: 200

Each Parquet file contains mixed, unordered IDs

⚠ Query must scan ALL files

STEP 2 ZORDER BY (id)

OPTIMIZE rewrites all files sorted by id

0– 124K	125K– 249K
250K– 374K	375K– 499K
500K– 624K	625K– 749K
750K– 874K	875K– 1M

Files fully sorted by id after OPTIMIZE runs

✓ Query skips irrelevant files

⚡ Full table rewrite every time

STEP 3 CLUSTER BY (country)

Changing the key → full table rewrite

DE DE (8)	DE DE (12)
FR FR (3)	FR FR (9)
GB UK (5)	GB UK (11)
ES ES (2)	ES ES (7)

All files rewritten — sorted by country, not by id anymore.

⚡ Full rewrite — all files replaced

✓ New clustering key registered in log

STEP 4 Append + OPTIMIZE

Score-based rewrite: new files + affected neighbors

Score before OPTIMIZE:

0.55 — below threshold

DE DE (8)	DE DE (12)
FR FR (3)	FR FR (9)
GB UK ★	ES ES ★
GB UK (5)	ES ES (2)

★ = rewritten (new files + degraded neighbors)

Score after OPTIMIZE:

0.87 — restored

✓ Incremental — only degraded bins rewritten

Automatic Liquid Clustering

CLUSTER BY AUTO — Databricks selects clustering keys for you

How it works

Databricks analyzes the table's historical query workload and automatically selects the most effective clustering columns — no manual tuning required.

Requirements

Predictive Optimization must be enabled
Unity Catalog managed tables only

✓ When to use AUTO

Ideal when query patterns are unclear or evolving.
Use explicit `CLUSTER BY (col)` when you know your access patterns well.

Syntax

New tables

```
CREATE TABLE employees(  
  id INT, country STRING, salary INT  
)  
CLUSTER BY AUTO
```

Existing tables

```
ALTER TABLE employees  
CLUSTER BY AUTO;  
  
-- Then run OPTIMIZE to apply  
OPTIMIZE employees;
```

Partitioning vs Z-Order vs Liquid Clustering

Partitioning

Divide físicamente los datos en subdirectorios por el valor de una columna. Ideal para columnas de baja cardinalidad con filtros exactos.

Casos concretos

Z-Order Indexing

Ordena los datos dentro de los ficheros Parquet por una o varias columnas. Requiere OPTIMIZE completo en cada ejecución.

Casos batch

Liquid Clustering

Organización flexible a nivel de tabla. OPTIMIZE incremental basado en clustering score. Cambia de clave sin reescribir.

Default moderno

Característica	Partitioning	Z-Order Indexing	Liquid Clustering
Nivel de definición	Directorios	Ficheros	Tabla
Crea subcarpetas	✓	✗	✗
OPTIMIZE incremental	✗	✗	✓
Requiere OPTIMIZE	✗	✓	✓
Reorganiza tabla completa	✗	▲ Puede	✗
Alta cardinalidad	▲ Mala opción	Buena opción	Buena opción
Data skipping	✓	✓	✓
Small files problem	▲ Frecuente	✗	✗
Cambio de clave	✗ Reescritura	▲ Reoptimización	✓ Sin reescritura
Escala en tablas grandes	▲	▲	✓
Recomendado hoy	▲ Casos concretos	▲ Casos batch	✓ Default moderno

 Liquid Clustering es el estándar recomendado por Databricks para tablas nuevas. Partitioning sigue siendo válido para casos de retención o compliance. Z-Order está en proceso de deprecación.

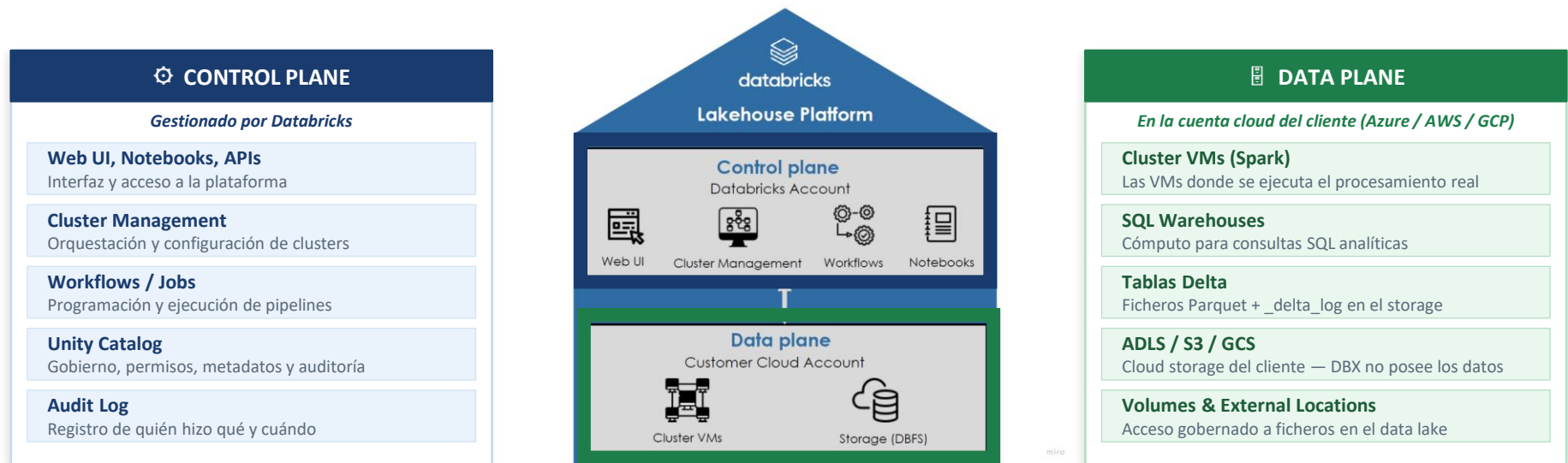
DEMO 1.3.2
ADVANCE DELTA FEATURES II

METASTORE & TABLES

1. Control Plan vs Data Plan
2. Where is the Data & Metadata
3. External Tables vs Manage Tables
4. Personas, Permissions and Workspace Objects

Control Plane vs Data Plane I

Databricks separa la gestión de la plataforma de los datos y el procesamiento. Esta distinción es clave en arquitecturas cloud modernas para garantizar seguridad, escalabilidad y soberanía del dato.



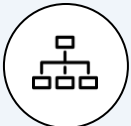
💡 Databricks gestiona el gobierno — el cliente mantiene la propiedad y control de sus datos

Control Plane vs Data Plane II

Control Plane



Metadatos



Esquema



Definición Tabla



Formato Datos



Ubicación



Particionado



Permisos

Data Plane



Datos



Parquet



JSON



CSV



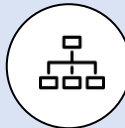
Excel

Zoom en Metadatos

Metadatos

En Databricks, los **metadatos** son la información que **describe y define los datos**, pero **no son los datos en sí**. Su función es permitir que el sistema entienda **qué es una tabla, cómo debe leerse y dónde se encuentran físicamente sus datos**, sin necesidad de acceder a los archivos.

Los metadatos se almacenan en el **metastore** (Hive Metastore o Unity Catalog) y actúan como un **catálogo central** de todas las entidades de datos.



Esquema

- Nombre columnas
- Tipo de dato
- Orden columnas
- Nulable / No Nulable
- Comentarios (si existen)



Definición Tabla

- Nombre de la tabla
- Base de datos a la que pertenece
- Tipo de tabla (Managed, external)



Formato Datos

- PARQUET
- CSV
- JSON
- DELTA



Ubicación

- Ruta física de los datos



Particionado



Permisos

- Quien es el owner
- Que grupos pueden hacer select
- Quien puede modificar el schema
- Quien puede borrar la tabla

External Tables vs Managed Tables

Managed Tables



Concept

- Unity Catalog owns both metadata and data lifecycle.



Steps

- > Create Table



Advantages

- Clean Lifecycle Management
- Automatic Cleanup & Simpler governance
- *Safer for curates layers (Silver/Gold)
- Lower Risk on orphaned Data



Risks

- DROP = irreversible physical deletion
- Not ideal if storage is shared with other systems



Ideal for...

- Internal lakehouse tables
- Transformed Data
- Production Data Marts
- Controlled Enterprise environments

External Tables

- Unity Catalog owns metadata only. Storage lifecycle is external.

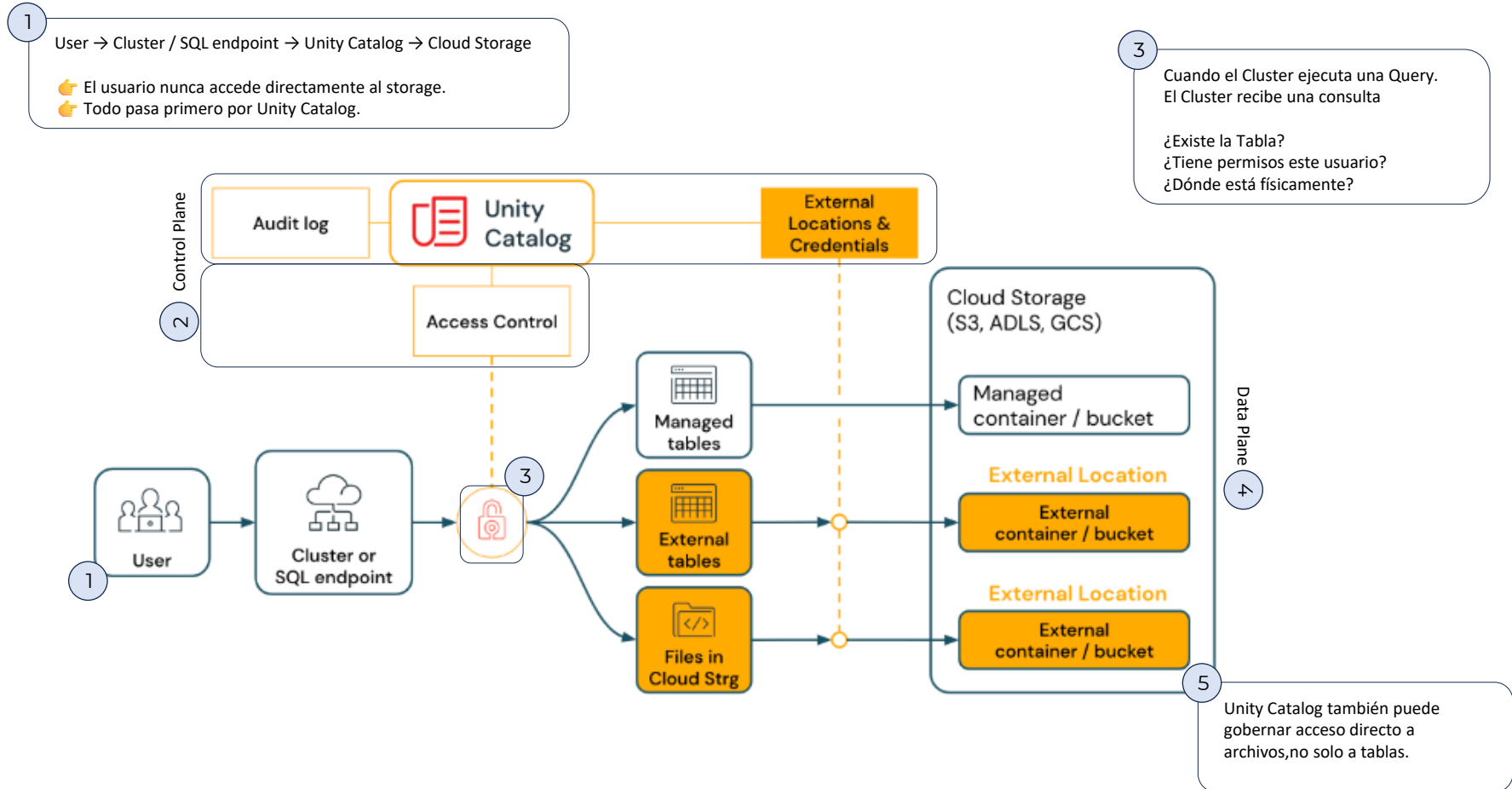
- > Create Storage Credential
- > Create the external Location
- > Grant Permission on the external Location
- > Create Table

- Data survives table deletion
- Good for shared storage
- Works well for ingestion zones
- Useful for migrations
- Allows multi-engine Access (Spark, Synapse, Presto)

- Orphaned data if not managed properly
- Manual cleanup required
- If storage changes externally, table can break

- Raw landing zone
- Cross-Platform data sharing
- Data owned by other team
- External Integrations

External Tables vs Managed Tables – Flujo General



DEMO 1.4
EXTERNAL & MANAGED TABLE

SET UP DELTA TABLE

1. CTAs statements
2. Table Constraints
3. Cloning Delta Lake Tables
4. Views in Databricks

CREATE TABLE vs CTAS

Dos formas de crear una tabla Delta — una vacía con schema explícito, otra poblada desde una query



CREATE TABLE

Crea una tabla vacía con un schema definido explícitamente por el desarrollador. Los datos se insertan después mediante INSERT INTO u otras operaciones de escritura.

```
CREATE TABLE employees (  
  id      INT      NOT NULL,  
  name    STRING    ,  
  salary  DOUBLE  
) USING DELTA;
```

- ✓ Schema declarado explícitamente — control total sobre tipos y nombres
- ✓ Tabla creada vacía — sin datos hasta el primer INSERT
- ✓ Compatible con constraints (NOT NULL, CHECK)
- ✓ Permite definir PARTITIONED BY, CLUSTER BY, LOCATION...
- ✗ Requiere INSERT INTO explícito para poblar la tabla

Usar cuando: necesitas control preciso del schema, constraints, o crear la tabla antes de que lleguen los datos.



CTAS (Create Table As Select)

Crea una tabla y la puebla en una sola operación a partir del resultado de una query SELECT. El schema se infiere automáticamente de las columnas del SELECT.

```
CREATE TABLE employees_eu  
USING DELTA  
CLUSTER BY (id)  
AS SELECT id, name, salary  
FROM employees  
WHERE region = 'EU';
```

- ✓ Schema inferido automáticamente del SELECT — sin declaración manual
- ✓ Tabla creada ya poblada con datos en una sola operación
- ✓ Acepta opciones Delta: CLUSTER BY, LOCATION, TBLPROPERTIES...
- ✗ No soporta constraints (NOT NULL, CHECK) en la misma instrucción
- ✗ No permite PARTITIONED BY junto con CLUSTER BY

Usar cuando: quieres crear una tabla derivada de otra existente — transformaciones, filtros, migraciones o ejercicios de clustering.

	Schema explícito	Tabla con datos	Soporta constraints	Compatible CTAS+CLUSTER BY
CREATE TABLE	✓	✗	✓	✓
CTAS	✗	✓	✗	✓

Table Constraints

Reglas de integridad aplicadas en escritura — **se almacenan en el transaction log de Delta**

 NOT NULL	 CHECK	 DROP CONSTRAINT
Garantiza que ningún valor de la columna sea NULL. Se aplica en INSERT y UPDATE — cualquier intento de escribir un NULL falla con error.	Define una condición booleana que toda fila debe cumplir. Cualquier escritura que produzca filas donde la condición sea FALSE o NULL es rechazada.	Elimina un constraint existente por nombre. No valida los datos existentes — solo deja de aplicar la regla en escrituras futuras.
<pre>ALTER TABLE employees ALTER COLUMN salary SET NOT NULL;</pre>	<pre>ALTER TABLE employees ADD CONSTRAINT valid_salary CHECK (salary > 0 AND salary < 500000);</pre>	<pre>ALTER TABLE employees DROP CONSTRAINT valid_salary;</pre>
 Falla si ya existen NULLs en la tabla al aplicar el constraint	 El nombre del constraint es obligatorio — se usa para eliminarlo después	 Los datos existentes que violaban el constraint no se modifican

	Se aplica en escritura	Falla si datos violan regla	Se guarda en Delta log	Eliminable por nombre
NOT NULL	✓	✓	✓	✓
CHECK	✓	✓	✓	✓
Primary Key*	✗	✗	✗	✗
Foreign Key*	✗	✗	✗	✗

* Primary Key y Foreign Key existen como sintaxis en Delta Lake pero son informativos — no se refuerzan en tiempo de escritura.

 **Importante:** los CHECK constraints en Delta Lake se aplican solo en escrituras nuevas. Los datos históricos que ya violaban la regla **no se validan ni eliminan** al añadir el constraint. En Not null si que fallaría.

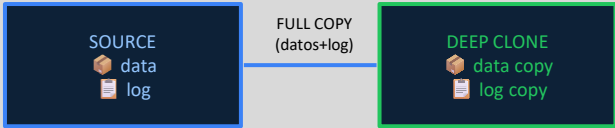
Deep Clone vs Shallow Clone

Dos formas de copiar una tabla Delta — misma sintaxis, comportamiento radicalmente diferente

Deep Clone

¿Qué hace?

Copia físicamente todos los ficheros Parquet + el transaction log (`_delta_log`) a una nueva ubicación. El clon es completamente independiente de la fuente.



✓ Independiente — cambios en uno no afectan al otro

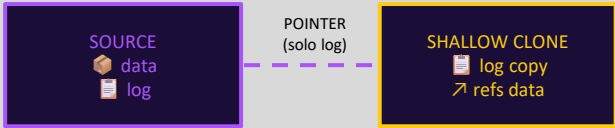
```
CREATE TABLE clone_table DEEP CLONE source_table;
```

- ✓ Clon completamente independiente de la fuente
- ✓ Soporta sincronización incremental (re-ejecutar CLONE)
- ✓ Funciona aunque se haga VACUUM en la fuente
- ✗ Coste alto: copia todos los datos físicamente
- ✗ Lento en tablas grandes

Shallow Clone

¿Qué hace?

Solo copia el transaction log. Los ficheros Parquet NO se duplican — el clon apunta a los mismos ficheros que la fuente. Casi instantáneo y sin coste de almacenamiento.



⚠ Depende de que los ficheros fuente no sean eliminados (VACUUM)

```
CREATE TABLE clone_table SHALLOW CLONE source_table;
```

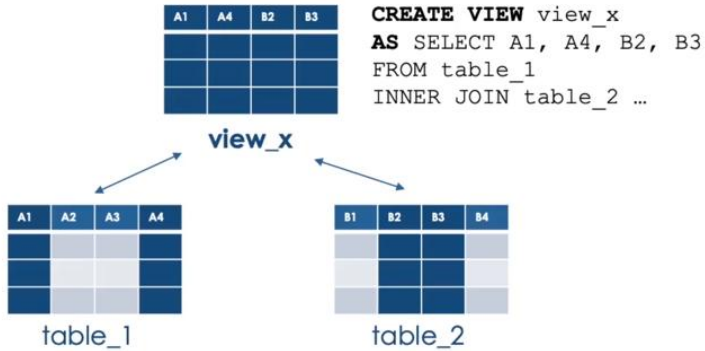
- ✓ Instantáneo — no copia datos físicos
- ✓ Coste de almacenamiento casi cero
- ✓ Ideal para entornos de dev/test y exploración rápida
- ✗ Se rompe si la fuente hace VACUUM (borra ficheros referenciados)
- ✗ No soporta sincronización incremental

En ambos casos: los cambios en el clon **nunca** afectan a la tabla fuente. **Cuándo usar cada uno** → **Deep Clone** para backups, migraciones y copias de larga duración. **Shallow Clone** para pruebas, desarrollo y pipelines CI/CD de corta duración.

Views en Databricks

Tres tipos de vista — misma sintaxis SELECT, ciclo de vida radicalmente diferente

=> Logical Query against source tables



💡 Una vista **NO** almacena datos. Es una query guardada — cada vez que se consulta, se ejecuta sobre las tablas fuente en tiempo real.

VIEW (Estándar)

Vistas de negocio reutilizables y compartidas entre equipos

```
CREATE VIEW my_view AS SELECT ...;
```

- ✓ Persiste en el metastore hasta que se elimina explícitamente
- ✓ Visible para todos los usuarios con permisos sobre el schema
- ✓ Compatible con Unity Catalog — soporta permisos granulares
- ❗ No almacena datos — ejecuta la query en cada consulta

TEMP VIEW (Temporal)

Transformaciones intermedias dentro de un notebook o pipeline

```
CREATE TEMP VIEW my_view AS SELECT ...;
```

- ✓ Scoped a la sesión activa de Spark — desaparece al cerrar
- ✓ No requiere permisos especiales ni Unity Catalog
- ✗ No visible para otros usuarios ni otras sesiones
- ❗ Útil para transformaciones intermedias en notebooks

GLOBAL TEMP VIEW

Compartir resultados intermedios entre notebooks en el mismo cluster

```
CREATE GLOBAL TEMP VIEW my_view ...
```

- ✓ Compartida entre sesiones del mismo cluster — prefijo global_temp.
- ✓ Persiste mientras el cluster esté activo
- ✗ Se elimina cuando el cluster se reinicia o para
- ❗ Acceso: SELECT * FROM global_temp.my_view

`DROP VIEW view_x;` | Las vistas estándar persisten tras reiniciar el cluster. | Las TEMP y GLOBAL TEMP no requieren DROP — desaparecen solas.

Ciclo de vida de las Views

Para entender cuándo vive cada tipo de vista, primero hay que entender cuándo vive una sesión y un cluster

Cluster

Empieza: cuando lo arrancas manualmente, o cuando Databricks lo lanza automáticamente al ejecutar la primera celda.

Termina: cuando lo paras manualmente, cuando se alcanza el timeout de inactividad (auto-termination), o cuando Databricks lo recicla.

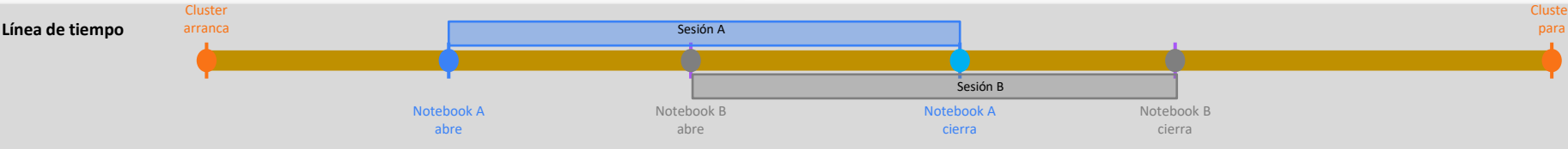
Scope: todos los notebooks y sesiones que se conecten a ese cluster comparten el mismo proceso Spark.

Sesión de Spark (SparkSession)

Empieza: cuando abres un notebook y ejecutas la primera celda — Databricks crea una SparkSession automáticamente (objeto spark).

Termina: cuando el notebook se desconecta del cluster, se cierra la pestaña del navegador, o se llama a spark.stop().

Scope: cada notebook tiene su propia sesión — dos notebooks en el mismo cluster tienen sesiones distintas e independientes.



VIEW (Estándar)

Scope: Metastore

Persiste indefinidamente

- ✓ Sobrevive al cierre del notebook
- ✓ Sobrevive al reinicio del cluster
- ✓ Sobrevive al cierre del navegador
- ✗ Solo desaparece con DROP VIEW
- ✓ Visible para todos los usuarios con permisos

```
CREATE VIEW my_view AS SELECT ...;
```

Usar cuando: Vistas de negocio permanentes, compartidas entre equipos

TEMP VIEW

Scope: SparkSession (notebook)

Vive mientras la sesión esté activa

- ✗ Muere cuando el notebook se desconecta
- ✗ Muere cuando cierras la pestaña del navegador
- ✗ Muere si el cluster se reinicia
- ❗ NO visible desde otros notebooks
- ❗ No requiere DROP — desaparece sola

```
CREATE TEMP VIEW my_view AS SELECT ...;
```

Usar cuando: Transformaciones intermedias en un notebook o pipeline

GLOBAL TEMP VIEW

Scope: Cluster (base de datos global_temp)

Vive mientras el cluster esté activo

- ✓ Compartida entre todos los notebooks del cluster
- ✗ Muere cuando el cluster se para o reinicia
- ❗ Acceso obligatorio con prefijo global_temp.
- ✓ Visible desde cualquier sesión del mismo cluster
- ❗ No requiere DROP — muere con el cluster

```
SELECT * FROM global_temp.my_view
```

Usar cuando: Compartir datos entre notebooks del mismo cluster

DEMO 1.5

VIEWS IN DATABRICKS

DATA PROCESING & TRANSFORMATION

1. Querying Files Directly
2. Extract Files as Raw Content
3. Configure Options for External Sources
4. CTAS from Files

Learning Objectives

Ingesta de datos externos en Databricks

01 Querying Files Directly

Consultar ficheros en S3/ADLS directamente con SQL usando la sintaxis `format.`/path/to/file`` — sin registrar ninguna tabla.

02 Extract Files as Raw Content

Extraer el contenido bruto de ficheros usando `text.` y `binaryFile.` — útil para ficheros de texto, logs o binarios.

03 Configure Options for External Sources

Registrar tablas no-Delta sobre ficheros externos con `CREATE TABLE ... USING ... OPTIONS ... LOCATION.`

04 CTAS from Files

Usar CTAS (CREATE TABLE AS SELECT) para crear tablas Delta permanentes a partir de ficheros externos en una sola operación.

Todos los métodos de este bloque leen datos externos — la diferencia clave es si el resultado es una tabla Delta registrada o una consulta puntual sobre el fichero.

Querying Files Directly

Consultar ficheros en cloud storage sin registrar ninguna tabla

`SELECT * FROM file format. /path/to/file`

Self-describing formats

- json
- parquet
- ...

Non self-describing Formats

- CSV
- TSV
- ...

Single file file_2022.json

Multiple files file_*.json

complete directory /path/dir

file_format — Formatos soportados

json	Infiere schema automáticamente	parquet	Infiere schema + columnar
delta	Transaccional, schema en log	csv	Requiere OPTIONS para separador, header...
tsv	Como CSV con tabulador	text	Cada línea como string
binaryFile	Contenido binario + metadata	avro / orc	Formatos columnar alternativos

`/path/to/file` — Opciones de ruta

Fichero único:	json.`/data/file_2022.json`	Lee exactamente ese fichero
Wildcard (múltiples ficheros):	json.`/data/file_*.json`	Todos los ficheros que coincidan con el patrón
Directorio completo:	parquet.`/data/sales/`	Todos los ficheros del directorio (mismo formato)
Cloud Storage (S3 / ADLS):	csv.`s3://bucket/path/`	Rutas absolutas en cloud — requiere permisos
DBFS:	delta.`dbfs:/FileStore/data/`	Sistema de ficheros de Databricks

⚠ Estas queries NO crean ninguna tabla en el metastore. Son consultas ad-hoc directamente sobre el fichero — cada vez que la ejecutas, Spark va a leer el fichero. Para persistir el resultado, usa CTAS.

📦 Extraer contenido bruto (Raw Content)

text → devuelve columna value STRING con el contenido de cada línea del fichero

binaryFile → devuelve path, modificationTime, length, content (BINARY) — útil para imágenes, PDFs, ficheros propietarios

Registering Tables on External Sources

Tablas no-Delta sobre ficheros externos — el acceso persiste aunque el fichero cambie

```
CREATE TABLE sales_csv
  (order_id INT, amount DOUBLE, country STRING)
USING CSV
OPTIONS (
  header = 'true',
  sep = '|'
)
LOCATION 's3://bucket/sales/';
```

USING

Formato del fichero: CSV, JSON, PARQUET, JDBC, DELTA...

OPTIONS

Parámetros específicos del formato (header, sep, inferSchema, url, dbtable...)

LOCATION

Ruta en cloud storage o DBFS donde están los ficheros. La tabla apunta a esa ruta — no copia los datos.

Non-Delta

No tiene transaction log, no soporta Time Travel, MERGE, ni las garantías ACID de Delta Lake.

External Table

Los datos NO se mueven — la tabla es solo un puntero al LOCATION. Si borras la tabla, los ficheros siguen en S3/ADLS.

Schema explícito

A diferencia del CTAS, aquí defines el schema manualmente. Necesario para formatos como CSV que no lo infieren solos.

Sin snapshot

Cada query lee los ficheros en tiempo real desde el LOCATION — si alguien añade ficheros al directorio, la tabla los ve automáticamente.

CTAS: Registering Delta Tables from Files

Convertir ficheros externos en tablas Delta permanentes en una sola operación

```
CREATE TABLE employees_delta
USING DELTA
CLUSTER BY (id)
AS SELECT * FROM parquet.`s3://bucket/raw/employees/`;
```

¿Qué hace exactamente?

1. Ejecuta el SELECT sobre los ficheros externos
2. Convierte el resultado a formato Delta
3. Escribe los datos en el LOCATION de la tabla
4. Registra la tabla en el metastore (Unity Catalog)

Comparativa de enfoques

	Query directa fmt. `/path`	External Table USING+OPTIONS	CTAS → Delta
Registra tabla en metastore	X	✓	✓
Es tabla Delta	X	X	✓
Datos copiados físicamente	X	X	✓
Soporta Time Travel	X	X	✓
Soporta MERGE / CDC	X	X	✓
Schema explícito	X	✓	X
Soporta OPTIONS del formato	X	✓	X
Resultado persiste	X	✓	✓

⚠ Limitaciones del CTAS

- X No soporta schema manual — el schema se infiere del SELECT

X No soporta OPTIONS del fichero fuente — usa query directa + CTAS si necesitas opciones
- X No soporta PARTITIONED BY junto con CLUSTER BY

✓ Sí acepta TBLPROPERTIES, LOCATION, CLUSTER BY, COMMENT

DEMO 2.1

QUERYING FILES

DEMO 2.2

QUERYING FILES